Executive Summary & Research Proposal

1. Project Title & Team

AdaptivePool: A Queue-Aware Adaptive Thread Pool for a Concurrent Web Server

OSTEP Project G.5, Concurrent Web Server

Name	Email
Yang (Selena) Zhang	[your Northeastern email]
[Teammate 2]	[email]
[Teammate 3]	[email]

2. Abstract

Web servers that use a fixed-size thread pool have to pick the pool size ahead of time. If the pool is too small, a sudden burst of requests piles up in the queue and tail latency shoots up. If it is too large, most threads sit idle when traffic is low and still compete for the CPU and locks when it is high. We will build a multi-threaded C web server based on the OSTEP starter code, where a master thread accepts connections and puts them into a bounded buffer that a pool of worker threads drains, with FIFO and Smallest-File-First scheduling as our baseline. On top of that, we will add AdaptivePool, a monitor thread that watches the queue length and grows or shrinks the worker pool between a minimum and a maximum size. We will write our own load generator that produces synthetic requests with steady and bursty arrival patterns and different mixes of CPU-heavy, I/O-heavy, short and long jobs, and use it to compare AdaptivePool against fixed pools of 1, 2, 4 and 8 threads. Our main measure is p99 latency, along with throughput, total completion time, and how many threads each design actually keeps alive.

3. The Research Extension (Hypothesis)

Mechanism. We will replace the fixed worker count with a feedback controller that changes the pool size at runtime.

- A **monitor thread** wakes up every sampling interval (about 50 ms), reads the current queue length q and the number of idle workers, and logs them.
- **Grow fast:** if q stays above a high watermark for k samples in a row and the pool is below its maximum, it creates a new worker with `pthread_create`.
- **Shrink slowly:** if the queue has been empty and some workers have been idle for a longer window, and the pool is above its minimum, it raises a `retire_target` counter

and broadcasts on the `not_empty` condition variable. An idle worker that wakes up, finds no work and sees `retire_target > 0` decrements it and exits. Workers never get killed in the middle of a request.
- Using different thresholds and windows for growing and shrinking (hysteresis) keeps the pool from flapping up and down.

This changes the core concurrency structure of the server: the pool goes from a static array of threads to a resizable set of workers, with a new pool-state structure protected by its own mutex and a safe retirement protocol. The idea of resizing a thread pool based on queue length comes from the SEDA architecture [1]. Our goal is to see how well a simple version of it works inside a small OSTEP-style server under the specific traffic patterns we care about.

Hypothesis. Under bursty traffic, AdaptivePool will get much lower p99 latency than a fixed pool that has the same *average* number of threads, and will get close to the p99 of the largest fixed pool (8 threads) while keeping fewer threads alive on average. Under steady traffic, we expect it to perform about the same as the best fixed pool, which would show that the monitor thread adds little overhead.

4. Research Question

Under high-volatility (bursty) traffic, does a queue-aware adaptive thread pool reduce p99 request latency compared to fixed-size thread pools, and how many worker threads does it use to get there?

- **Primary metric:** p99 end-to-end latency [2], measured from when the request is sent to when the response is received.
- **Secondary metrics:** p50 latency, throughput (completed requests per second), total completion time for a batch (makespan), queueing delay (`t_dequeue − t_arrive`), and average number of live workers over time (thread-seconds).
- **Independent variables:** pool type (fixed 1 / 2 / 4 / 8 vs. adaptive with min 1, max 8), arrival pattern (steady Poisson vs. bursty on/off), request mix (percent of CPU-heavy vs. I/O-heavy requests), and job-length distribution (uniform vs. heavy-tailed). Target load is about 1000 requests per second on our VM.

5. Proposed Scope

1. **Baseline concurrent server.** Turn the single-threaded OSTEP `wserver` [4] into a master/worker design with a bounded buffer, one mutex and two condition variables (`not_empty`, `not_full`), no busy waiting. Support `-t threads -b buffers -s FIFO|SFF` as the project spec requires (SFF is a simple form of size-based scheduling [3]), and block path traversal (`..`).

2. **Workload generator.** Write an open-loop client that sends requests at a target rate with steady (Poisson) or bursty (on/off) arrivals. It mixes static files of different sizes, an I/O-bound CGI (sleeps) and a CPU-bound CGI (busy loop), and writes per-request timestamps and latency to CSV.
3. **AdaptivePool.** Implement the monitor thread, the grow/shrink policy with hysteresis, and safe worker retirement, with min, max, watermarks and sampling interval set from the command line.
4. **Instrumentation.** Record `t_arrive`, `t_dequeue` and `t_done` for each request, and sample queue length and number of live workers over time with low overhead.
5. **Experiments and analysis.** Run the full matrix of fixed vs. adaptive pools × steady/bursty traffic × CPU/I/O mixes, repeat each run several times, and plot latency percentiles, throughput and thread count over time to answer the research question.

References

[1] M. Welsh, D. Culler, and E. Brewer. "SEDA: An Architecture for Well-Conditioned, Scalable Internet Services." SOSP 2001.

[2] J. Dean and L. A. Barroso. "The Tail at Scale." Communications of the ACM, 2013.

[3] M. Harchol-Balter, B. Schroeder, N. Bansal, and M. Agrawal. "Size-Based Scheduling to Improve Web Performance." ACM Transactions on Computer Systems, 2003.

[4] R. and A. Arpaci-Dusseau. OSTEP Projects, "Concurrent Web Server." github.com/remzi-arpacidusseau/ostep-projects